

Homework - 2 Report

Name: Faruk KAPLAN
ID: 21050111026

Project Summary & Implementation Details:

-> Explanation of sequential.c file:

1-) Take input files from command line, then read them into matrices

2-) Allocate necessary memory

3-) Multiply the matrices. Pseudocode is:

```

Procedure multiply(A, x, result, row_A, col_A, row_x, col_x)
  For i = 0 to row_A - 1
    For j = 0 to col_x - 1
      result[i][j] = 0.0
      For k = 0 to col_A - 1
        result[i][j] = result[i][j] + A[i][k] * x[k][j]
      End For
    End For
  End For
End Procedure

```

4-) Print the elapsed time and write y matrix into y.txt

-> Explanation of point-to-point.c file:

1-) Take input files from command line, then read them into matrices

2-) Flatten the A matrix for split

3-) Send the processes to its portion, only using MPI_Send()

```

For i = 1 to proc - 1
  For j = 0 to row_process * col_A - 1
    tempMatrix[j] = flattened_A[(i * row_process * col_A) + j]
  End For
  MPI_Send(tempMatrix, row_process * col_A, MPI_DOUBLE, i, 0, MPI_COMM_WORLD)
End For

```

4-) Each process does the multiplication with its own portion

```

Function multiply(portion_A, x, row, col)
  result = AllocateMemory(row * sizeof(double))
  For i = 0 to row - 1
    tempResult = 0.0
    For j = 0 to col - 1
      tempResult = tempResult + portion_A[(i * col) + j] * x[j][0]
    End For
    result[i] = tempResult
  End For
  Return result
End Function

```

5-) Collect results, only using MPI_Recv(), and print it into y.txt

6-) Collect measured times from other processes and sum them

7-) Print the total elapsed time

-> Explanation of collective.c file:

1-) Take input files from command line, then read them into matrices

2-) Flatten the A matrix

3-) Scatter the flattened A matrix with MPI_Scatter()

4-) Do multiplication (same pseudocode as point-to-point.c 4th article)

5-) Gather the results with MPI_Gather() and print it into y.txt, Also print the total elapsed time

Results:

- > Machine: Macbook Pro M2 chip
- > Compiler: Mpicc
- > MPI Implementation: OpenMPI
- > Files: BigA.txt and BigX.txt

Core\Mode	Sequential	Point-to-point	Collective
1	3.64 ms	3.61 ms	3.5 ms
2		3.7 ms	1.9 ms
4		3.96 ms	0.9 ms

Discussion:

- > Sequential and collective mode is as expected, when we increase number of cores the speed of code is also increase
- > In point to point communication when we increase number of cores the speed decreases but it should be increase also. Maybe it is because of the communication overhead, since there is too much communication between processes (because of the point to point communication) it consumes too much time for that
- > Maybe I can somehow decrease number of p2p communication then the performance may be get better, or maybe I can find more efficient multiplication algorithm.